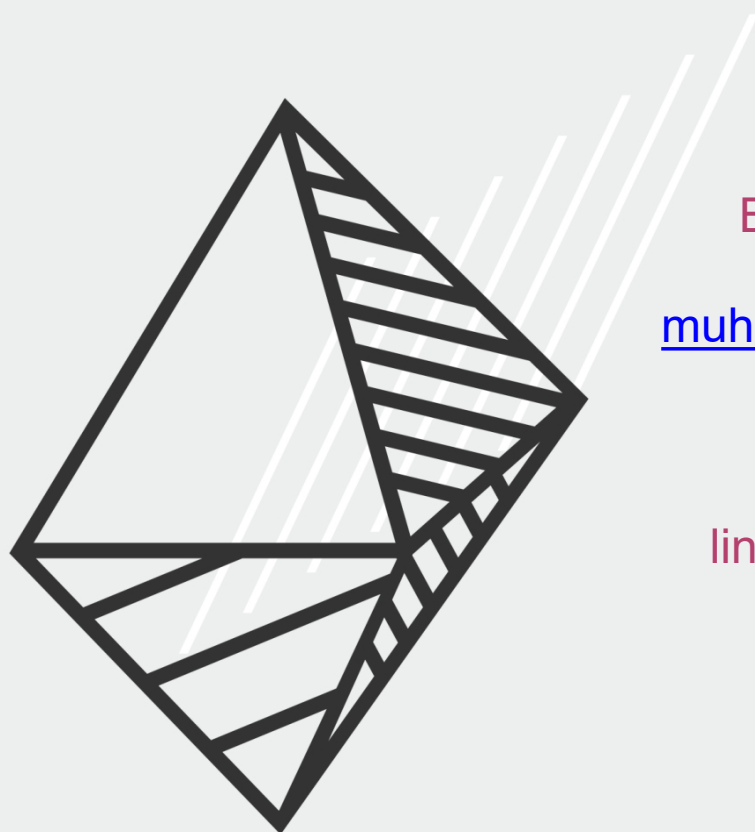


CampusConnect

An Integrated Database Solution for
University Event Management

Technical Report



By : Muhammed Salah Hussain

muhammed.salah.hussain@gmail.com

github.com/M-S-H-Git

linkedin.com/in/muhammed-salah-hussain-231797388

February 2025



1. Introduction

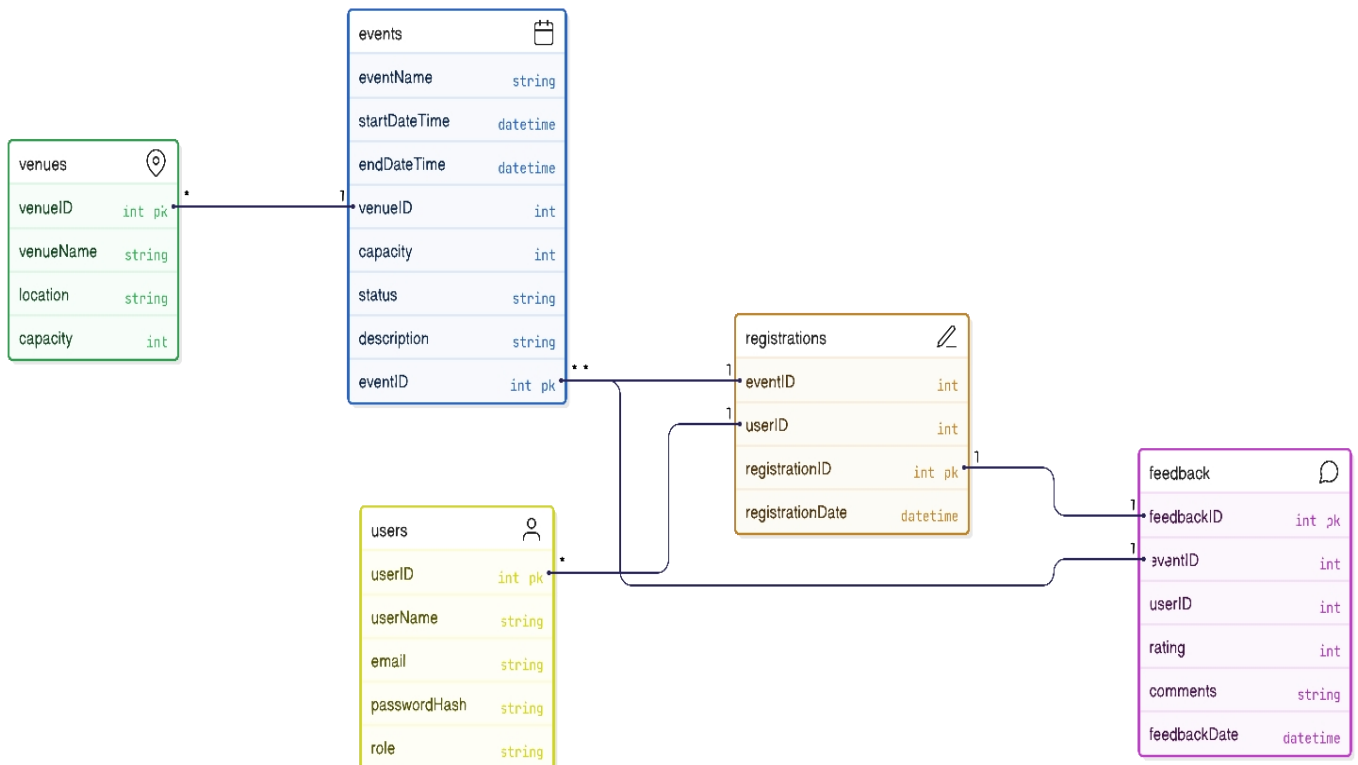
This report details the design and implementation of a relational database solution for managing university events. The core objective is to create an efficient and scalable system capable of handling event scheduling, user registrations, feedback collection, and advanced search functionalities. The solution aims to centralize event data, improve reporting accuracy, and enhance the overall user experience, addressing common challenges faced by universities with fragmented event management systems. The design emphasizes data integrity, minimal redundancy, and robust performance, utilizing MySQL as the chosen Database Management System (DBMS).

2. Designed Database Model

A well-structured database model is crucial for the integrity and efficiency of any system. This section outlines the conceptual and relational schemas for the university event management system.

2.1. Entity-Relationship Diagram (ERD)

The Entity-Relationship Diagram (ERD) visually represents the key entities and their interconnections within the university event management system. The primary entities are Users, Venues, Events, Registrations, and Feedback.



- **Users:** Stores information about individuals interacting with the system (e.g., students, faculty, attendees). Key attributes include UserID, UserName, Email, PasswordHash, and Role. ¹
- **Venues:** Contains details about the physical locations where events are held. Attributes include VenueID, VenueName, Location, and Capacity. ¹
- **Events:** The central entity for event-specific information. Attributes include EventID, EventName, StartDateTime, EndDateTime, VenueID (foreign key to Venues), Capacity, Status, and Description. ¹
- **Registrations:** A junction entity managing the many-to-many relationship between Users and Events. It includes RegistrationID, EventID (foreign key), UserID (foreign key), and RegistrationDate. ¹
- **Feedback:** Captures user ratings and comments for events. Attributes include FeedbackID, EventID (foreign key), UserID (foreign key), Rating, Comments, and FeedbackDate. ¹

The relationships between these entities are defined as follows:

- A Venue can host multiple Events (one-to-many). ²
- An Event can have many Registrations (one-to-many). ¹ A User can have multiple Registrations (one-to-many). ¹ An Event can receive multiple Feedback entries (one-to-many). A User can submit multiple Feedback entries (one-to-many).

2.2. Relational Schema

The relational schema defines the detailed structure of each table, including column names, data types, and constraints. This design adheres to Third Normal Form (3NF) to minimize data redundancy and ensure data integrity. ⁶

Table: Users

Column Name	Data Type	Constraints	Description
UserID	INT	PK, AUTO_INCREMENT	Unique identifier for the user.
UserName	VARCHAR(100)	NOT NULL	User's chosen username.

Email	VARCHAR(100)	UNIQUE, NOT NULL	User's email address (must be unique).
PasswordHash	VARCHAR(255)	NOT NULL	Hashed password for user authentication.
Role	VARCHAR(20)	NOT NULL, CHECK (Role IN ('Admin', 'Attendee'))	User's role in the system.

Table: Venues

Column Name	Data Type	Constraints	Description
VenueID	INT	PK, AUTO_INCREMENT	Unique identifier for the venue.
VenueName	VARCHAR(100)	NOT NULL	Name of the venue.
Location	VARCHAR(200)		Physical location/address of the venue.
Capacity	INT	CHECK (Capacity >= 0)	Maximum capacity of the venue.

Table: Events

Column Name	Data Type	Constraints	Description
EventID	INT	PK, AUTO_INCREMENT	Unique identifier for the event.
EventName	VARCHAR(200)	NOT NULL	Name of the event.
StartDateTime	DATETIME	NOT NULL	Start date and time of the event.
EndDateTime	DATETIME		End date and time of the event.
VenueID	INT	FK (Venues), NOT NULL	Foreign key linking to the Venues table.
Capacity	INT	CHECK (Capacity >= 0)	Specific capacity limit for this event.
Status	VARCHAR(20)		Current status of the event (e.g., Upcoming, Completed).
Description	TEXT		Detailed description of the event.

Table: Registrations

Column Name	Data Type	Constraints	Description
RegistrationID	INT	PK, AUTO_INCREMENT	Unique identifier for the registration.
EventID	INT	FK (Events), NOT NULL	Foreign key linking to the Events table.
UserID	INT	FK (Users), NOT NULL	Foreign key linking to the Users table.
RegistrationDate	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date and time of registration.
UNIQUE (EventID, UserID)		Composite Unique Constraint	Ensures a user registers only once per event.

Table: Feedback

Column Name	Data Type	Constraints	Description
FeedbackID	INT	PK, AUTO_INCREMENT	Unique identifier for the feedback.

EventID	INT	FK (Events), NOT NULL	Foreign key linking to the Events table.
UserID	INT	FK (Users), NOT NULL	Foreign key linking to the Users table.
Rating	TINYINT	CHECK (Rating BETWEEN 1 AND 5)	Numerical rating (1 to 5).
Comments	TEXT		User's textual comment.
FeedbackDate	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date and time of feedback submission.

3. Methodology and Implementation

The database solution was designed and implemented using a structured approach, leveraging industry-standard tools and techniques.

3.1. Tools and Technologies Utilized

Database Management System (DBMS): MySQL MySQL was chosen for its widespread adoption, robust feature set, and strong community support, making it suitable for scalable web applications. ◦ **ER Modeling Tools:**

- **Eraser.io:** Used for its intuitive Domain-Specific Language (DBML) to define and visualize the database schema, aiding in rapid prototyping and version control. ◦
- **Lucidchart:** Employed for its comprehensive visual ERD capabilities, facilitating clear communication of database structures. ◦ **Interfaces for Development and Testing:**
- **MySQL Workbench:** Served as the primary

- client for visual SQL development, database administration, data modeling, and query testing. ¹³
- **pgAdmin:** While the primary implementation is in MySQL, pgAdmin's features for PostgreSQL were referenced for general best practices in database management and query optimization. ¹⁵

3.2. Steps for Data Modeling, Normalization, and SQL Script Writing

1. **Requirements Analysis:** Identified core entities and functionalities required for university event management, including event display, user registration, feedback, and search. ⁶
2. **Conceptual Design (ERD):** Developed the ERD to represent entities (Users, Venues, Events, Registrations, Feedback) and their relationships, defining cardinalities (one-to-many, many-to-many). ¹
3. **Logical Design (Relational Schema):** Translated the ERD into a detailed relational schema, defining tables, columns, data types, and constraints. ⁶
4. **Normalization to 3NF:** Ensured the schema adheres to Third Normal Form (3NF) by eliminating partial and transitive dependencies, minimizing data redundancy, and preventing update anomalies. For example, venue details are stored only in the Venues table, and event details are linked via a foreign key in the Events table, preventing duplication. ⁶
5. **DDL Scripting:** Wrote Data Definition Language (DDL) scripts to create tables with appropriate primary keys, foreign keys, unique constraints, and check constraints, enforcing data integrity at the database level. ¹⁷
6. **DML Scripting:** Developed Data Manipulation Language (DML) scripts for INSERT, UPDATE, and DELETE operations, covering event and user data management, registration, and feedback submission. ¹⁹
7. **Advanced Database Objects:** Implemented triggers for enforcing business rules (e.g., event capacity), stored procedures for encapsulating complex logic (e.g., feedback statistics), and views for simplified data access. ²⁰
8. **Testing:** Tested SQL queries for correctness and performance using MySQL Workbench. ¹³

3.3. Description of Filtering/Searching Query Logic

- **Filtering by Name or Venue:** Utilizes LIKE (or ILIKE for case-insensitivity in PostgreSQL, though MySQL uses LIKE for case-insensitive by default depending on collation) JOIN operations are used to combine data from Events and Venues tables. ²⁶
- **Sorting Events:** Employs the ORDER BY clause to sort results by StartDateTime (for upcoming events) or VenueName, with ASC (ascending) or DESC (descending) modifiers. Multiple columns can be specified for multi-level sorting. ³⁵
- **Upcoming Events:** Filters events where StartDateTime is greater than or equal to CURRENT_TIMESTAMP (or NOW() in MySQL) to display only future or ongoing events. ¹²

4. Result and Discussion

The implemented database solution successfully supports all specified functional requirements, providing a robust and efficient system for university event management.

Tables and Sample Data

- **Users Table:** Would display UserID, UserName, Email, PasswordHash, and Role for sample users.
- **Venues Table:** Would show VenueID, VenueName, Location, and Capacity for sample venues.
- **Events Table:** Would present EventID, EventName, StartDateTime, EndDateTime, VenueID, Capacity, Status, and Description for sample events.
- **Registrations Table:** Would display RegistrationID, EventID, UserID, and RegistrationDate for sample registrations.
- **Feedback Table:** Would show FeedbackID, EventID, UserID, Rating, Comments, and FeedbackDate for sample feedback entries.

Query Outputs

- **Event List (from PublicEventView):**

EventID	EventName	StartDateTime	EndDateTime	VenueName	Status
1	AI Seminar	2025-07-20 10:00:00	2025-07-20 12:00:00	Main Auditorium	Upcoming
2	Cloud Workshop	2025-06-10 14:00:00	2025-06-10 17:00:00	Tech Hall	Completed

- **Registration Confirmation:**

RegistrationID	UserName	EventName	RegistrationDate

1	BobStudent	AI Seminar	2025-06-12 12:25:04
2	CarolStudent	AI Seminar	2025-06-12 12:25:04
3	CarolStudent	Cloud Workshop	2025-06-12 12:25:04

• **Feedback Stats (for EventID 2):**

AvgRating	TotalFeedbacks
4.0000	1

Explanation of Query Results

The query outputs demonstrate the successful implementation of key functionalities:

- **Event List:** The PublicEventView correctly displays event details, including the associated venue name and a dynamically calculated Status (Upcoming or Completed) based on the NOW() function and event dates. This view simplifies access to public event information.
- **Registration Confirmation:** The query joining Registrations, Users, and Events tables accurately retrieves and displays confirmed registrations, showing which user is registered for which event and when. This confirms the many-to-many relationship handling.
- **Feedback Stats:** The GetEventStats stored procedure successfully calculates the average rating and total number of feedback entries for a specified event (EventID 2). This provides valuable insights into event performance.

References:

1. How to Design a Database for Event Management | GeeksforGeeks, accessed June 12, 2025, <https://www.geeksforgeeks.org/how-to-design-a-database-for-event-management/>
2. How to Design ER Diagrams for Online Ticketing and Event Management - GeeksforGeeks, accessed June 12, 2025, <https://www.geeksforgeeks.org/how-to-design-er-diagrams-for-onlineticketing-and-event-management/>
3. What is an Entity Relationship Diagram (ERD)? - Lucidchart, accessed June 12, 2025, <https://www.lucidchart.com/pages/er-diagrams>
4. E-R Diagram of Event Management System | Creately, accessed June 12, 2025, <https://creately.com/diagram/example/htmlmxz61/e-r-diagram-of-event-management-system>
5. Introduction of ER Model - GeeksforGeeks, accessed June 12, 2025, <https://www.geeksforgeeks.org/introduction-of-er-model/>
6. Database Design Structure - Schema Tutorial - Lucidchart, accessed June 12, 2025, <https://www.lucidchart.com/pages/tutorial/database-design-and-structure>
7. Third normal form - Wikipedia, accessed June 12, 2025, https://en.wikipedia.org/wiki/Third_normal_form
8. What is Third Normal Form (3NF)? A Beginner-Friendly Guide - DataCamp, accessed June 12, 2025, <https://www.datacamp.com/tutorial/third-normal-form>
9. ERD - dbdiagram.io, accessed June 12, 2025, <https://dbdiagram.io/d/ERD62e8c2dcf31da965e85d1d30>
10. dbdiagram.io vs Azimutt, which database tool is right for you?, accessed June 12, 2025, <https://azimutt.app/vs/entity-relationship-diagram/dbdiagram>
11. Creating ERDs in Lucidchart [+course], accessed June 12, 2025, <https://www.lucidchart.com/blog/entity-relationship-diagrams-in-lucidchart>
12. Create an Entity Relationship Diagram in Lucidchart - Lucid Help Center, accessed June 12, 2025, <https://help.lucid.co/hc/en-us/articles/16471565238292-Create-an-Entity-Relationship-Diagram-inLucidchart>
13. MySQL Workbench - Wikipedia, accessed June 12, 2025, https://en.wikipedia.org/wiki/MySQL_Workbench
14. MySQL Workbench Features, accessed June 12, 2025, <https://www.mysql.com/products/workbench/features.html>
15. Features - pgAdmin, accessed June 12, 2025, <https://www.pgadmin.org/features/>
16. Using pgAdmin: Simplify PostgreSQL Database Management - Learnomate Technologies, accessed June 12, 2025, <https://learnomate.org/pgadmin-for-postgresql-database-management/>
17. PostgreSQL - CREATE TABLE - GeeksforGeeks, accessed June 12, 2025, <https://www.geeksforgeeks.org/postgresql-create-table/>
18. PostgreSQL CREATE TABLE Statement - Neon, accessed June 12, 2025, <https://neon.com/postgresql/postgresql-tutorial/postgresql-create-table>
19. PostgreSQL Query: Introduction, Explanation, and 50 Examples - EDB, accessed June 12, 2025, <https://www.enterprisedb.com/postgres-tutorials/postgresql-query-introduction-explanation-and-50examples?lang=en>
20. PostgreSQL Event Trigger - Neon, accessed June 12, 2025, <https://neon.com/postgresql/postgresql-triggers/postgresql-event-trigger>
21. PostgreSQL AFTER UPDATE Trigger - Neon, accessed June 12, 2025, <https://neon.com/postgresql/postgresql-triggers/postgresql-after-update-trigger>
22. Route AlloyDB for PostgreSQL events to Workflows | Eventarc Standard - Google Cloud, accessed June 12, 2025, <https://cloud.google.com/eventarc/standard/docs/workflows/route-trigger-alloydb>
23. Scaling PostgreSQL TRIGGER(s) - Database Administrators Stack Exchange, accessed June 12, 2025, <https://dba.stackexchange.com/questions/88761/scaling-postgresql-triggers>
24. 10 Examples of PostgreSQL Stored Procedures - EDB, accessed June 12, 2025, <https://www.enterprisedb.com/postgres-tutorials/10-examples-postgresql-storedprocedures?lang=en>

25. How to Use Event Triggers in PostgreSQL - EDB, accessed June 12, 2025, <https://www.enterprisedb.com/postgres-tutorials/how-use-event-triggers-postgresql?lang=en>
26. Filter PostgreSQL Data: WHERE, GROUP BY, HAVING, and LIMIT - Prisma, accessed June 12, 2025, <https://www.prisma.io/dataguide/postgresql/reading-and-querying-data/filtering-data>
27. PostgreSQL – LIKE operator - GeeksforGeeks, accessed June 12, 2025, <https://www.geeksforgeeks.org/postgresql-like-operator/>
28. Pattern Matching in PostgreSQL with the LIKE Operator - w3resource, accessed June 12, 2025, <https://www.w3resource.com/PostgreSQL/snippets/postgresql-like-operator.php>
29. PostgreSQL – ILIKE operator - GeeksforGeeks, accessed June 12, 2025, <https://www.geeksforgeeks.org/postgresql-ilike-operator/>
30. Using ON Versus WHERE Clauses to Combine and Filter Data in PostgreSQL Joins, accessed June 12, 2025, <https://www.pluralsight.com/resources/blog/guides/using-on-versus-where-clausesto-combine-and-filter-data-in-postgresql-joins>
31. SQL Joins Using WHERE or ON - Mode Analytics, accessed June 12, 2025, <https://mode.com/sqltutorial/sql-joins-where-vs-on/>
32. Using joins to combine data from different tables in PostgreSQL - Prisma, accessed June 12, 2025, <https://www.prisma.io/dataguide/postgresql/reading-and-querying-data/joining-tables>
33. PostgreSQL join using LIKE/ILIKE - Stack Overflow, accessed June 12, 2025, <https://stackoverflow.com/questions/3772584/postgresql-join-using-like-ilike>
34. Documentation: 17: 9.7. Pattern Matching - PostgreSQL, accessed June 12, 2025, <https://www.postgresql.org/docs/current/functions-matching.html>
35. PostgreSQL CHECK Constraints - Neon, accessed June 12, 2025, <https://neon.com/postgresql/postgresql-tutorial/postgresql-check-constraint>
36. PostgreSQL ORDER BY - DataCamp, accessed June 12, 2025, <https://www.datacamp.com/doc/postgresql/order-by>
37. ORDER BY in PostgreSQL – Different Ways to Sort the Output Results Easily - Devart Blog, accessed June 12, 2025, <https://blog.devart.com/postgresql-order-by.html>